

1位解法

Rank	1
Team	James Day
Author	James Day
Topic ID	549801
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549801>

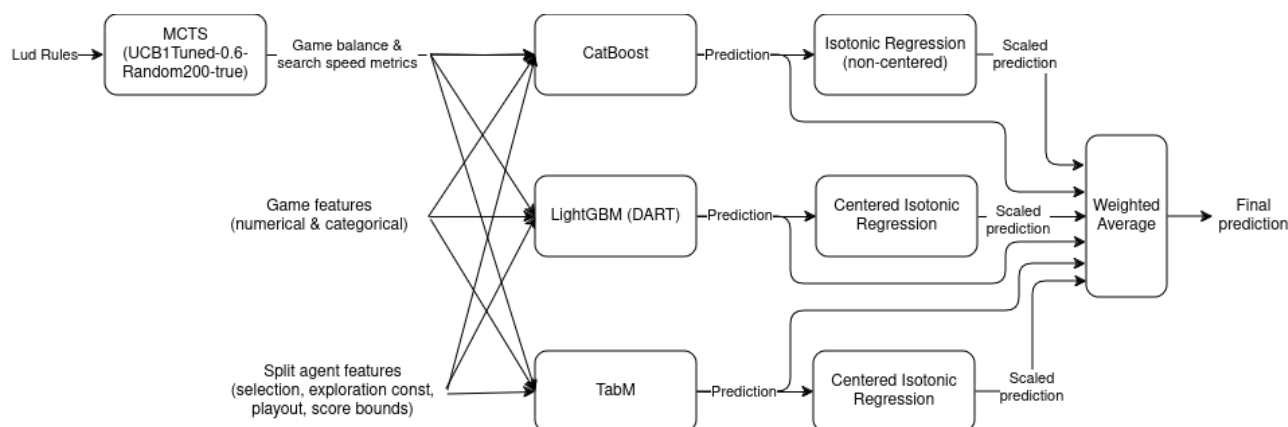
Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

KaggleとMaastricht Universityに、この興味深いコンペの開催を感謝します。多くを学びました。Leaderboard上位の皆さんにもお祝いを申し上げます。最後まで冷や汗ものでした！

概要

勝因となった最大の工夫は、各ゲームの初期局面で数秒間tree searchを実行し、ゲームの均衡度とsearch速度を表す追加特徴量を作ったことだと思います。これは主催者提供の AdvantageP1、MovesPerSecond、PlayoutsPerSecond と一部重複します。追加特徴量と他参加者も使った特徴量を、CatBoost、LightGBM、TabM、isotonic regressionのstacked ensembleへ入れました。



Base model + isotonic stack	CV	Public	Private
CatBoost	0.362	0.415	0.421
LightGBM	0.357	0.419	0.424
TabM	0.352	0.420	0.424
全model ensemble	0.344	0.413	0.417

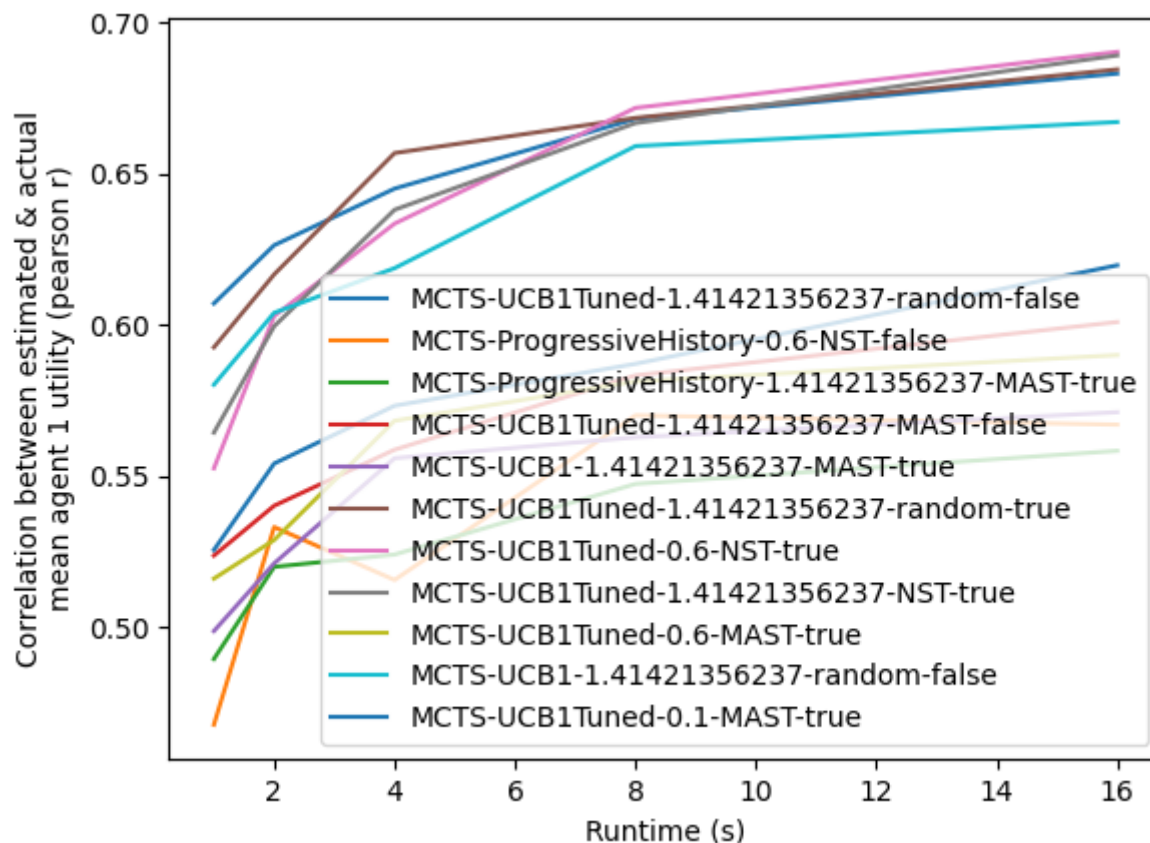
ensemble重みはCVに対してNelder-Meadで調整しました。CVはやや不安定でしたが、CV最良のensembleがPublic・Private双方でも最良になりました。

初期局面の評価

AdvantageP1 はrandom moveを行うagent同士のplayから計算されるため、非random playでplayer 1がどれほど有利かは完全には関連しません。非random agent同士のfull simulationは提出時のCPU負荷が高すぎ、ruleから有利さを予測するlanguage modelも精度不足でした。そこで初期局面から短いtree searchを行い、ゲーム均衡度の補助指標を作りました。非random agentの完全なsimulationと同程度の精度を約250倍高速に得られると考えています。速度と精度にはtrade-offがあり、15秒searchは非random agentを1時間対戦させる場合とおおむね同等でした。

初期実験では、各ゲーム15秒のtree searchで作った均衡指標により約0.045 CV、約0.012 LB改善し、コンペ中盤でPublic 1位へ上がりました。

使用したtree searchは、主催者が一部対局に使った UCB1Tuned-0.6-Random200-true と同等で、Ludiiの MCTS実装へlinkしています。この用途では selection と playout が重要で、exploration_const と score_bounds はあまり重要でないようでした。良かったconfigはUCB1/UCB1Tuned selectionとRandom/NST playoutの組合せです。ただしNSTはLeaderboardデータでtrainより悪く、UCB1TunedはCVでUCB1より少し良かったため、最終提出はUCB1Tuned + Randomを使いました。



Search速度の追加指標

主催者提供の MovesPerSecond と PlayoutsPerSecond にはrandom noiseがあります。初期局面評価と同時に、1秒当たりのaction数・search iteration数も記録しました。CVを約0.005改善したため最終解法で使いましたが、LBへの影響はrun間varianceの範囲内でした。

追加学習データ

追加の484 unique ruleset、合計14,365行を生成しました。

1. GAVELの独自実装：391 ruleset。
2. instruction-tuned LLM (Llama 3.1 70B、Qwen 2.5 32B) のfew-shot prompting：93 ruleset。

LLMでは1万超のgameを生成しましたが、品質問題で大半を捨て、大きくdownsampleしました。GAVELは balanceとstrategic depthの指標上で高品質でしたが生成が遅く、LLMは約95%がLudiiのcompile/runtime errorとなったもののplayable gameの生成速度は一桁違いました。

annotationのCPU時間を節約するため、各ordered agent pairにつきrulesetごと4試合だけ行い、A先手4試合とB先手4試合としました。そのためlabelは主催者データよりnoisyでしたが、約0.004 CV、0.001～0.002 LB改善しました。追加sampleを低weightにしてnoiseを緩和し、modelごとにweightをCV調整しました。

Model	追加データweight
CatBoost	0.25
LightGBM	1.0
TabM	0.5

追加データを増やすほど収穫逨減が激しく、最初の約7k行はCV約0.003改善、次は約0.001だけでした。量を増やすほど最適weightも下がり、label noiseが問題になったと考えます。今なら半分のgameへ2倍の計算を使う方が良かったと思います。

データ拡張

2種類を使いました。

1. tree-search特徴量を学習exampleごと10回計算。CV holdoutでない9 runから学習時の値をrandom選択し、ruleset内の全行へ同じ誤差を複製するのではなくnoiseへ意図的に曝露。CV/LBとも約0.002改善し相関も良好。
2. 提供データの非決定的特徴量を5回再計算し、「真の値」のより正確でnoiseの少ない推定を作成。学習時にはこの推定値と提供値／初期生成値の間をuniform sample。Ludii version discrepancyの疑いがある特徴量を落とすかで効果は変わりましたが、おおむね0.002以下の改善。

特徴量選択

主催者rulesetを再annotationすると、私のmachineではrun間で完全一致するのに主催者値とは異なる43特徴量がありました。Ludii player version差（私は1.3.13）が原因と推測しますが確証はありません。この43列をまとめて削除するとCVが一貫して0.001～0.002改善しました。他の選択法は複数seed・modelで一貫して改善せず、1列ずつでは効果が見えず、importance filteringでも有害列を見つけられませんでした。最終提出は疑わしい43特徴量をすべて不使用。終盤12時間で複数変更を同時投入したためLB単独効果は不明です。

Isotonic regression

公開Notebookの予測を係数倍して `[-1, 1]` より狭い範囲へclipする工夫に着想を得ました。CV調整したscale+clipはLBに害を与えましたが、OOB予測を特徴量とするisotonic regressionを後処理にするとCV/LB双方を約0.002改善しました。

大半のbase model stackにはcentered isotonic regressionを使いました。strict monotonicityにより過学習しにくいのか、scikit-learnの通常版よりCV小数第4位程度良好でした。ただしCatBoostでは `out_of_bounds='clip'` 相当機能がなくcrashしたため、scikit-learn版を使いました。推論前にraw予測を `[-1, 1]` へclipしても、学習予測rangeがそれより狭い場合は解決しません。

TabM

終盤にYandexがtabular deep learningのTabMと論文を公開しました。以前のdeep learning実験は有望でなかったものの、試すとLeaderboardでLightGBMと競合し、CV最良の単体modelになりました。以前のensembleへ追加した初期実験ではLB約0.001改善。最終寄与の正確な値にはablationが必要です。

ハイパーパラメータ調整

model種別ごとにOptunaを約6回、5-fold CV・1 seedで回し、候補を10-fold・3 seedで再確認しました。CVはruleset nameでgroup化したGroupKFoldShuffleです。

- TabMのcontinuous numerical feature向けpiecewise-linear embeddingが非常に有効で、通常MLPを上回る主因だと思います。
- TabM最良configは論文の探索rangeを少し外れ、wide hidden layer、small batch、low learning rateでした。
- LightGBMの `boosting='dart'` は重要で、単体を約0.004 CV、0.003 LB改善しましたが学習時間は5~10倍。探索はdart有無双方で実施しました。

アンサンブル

各model blockは実際には20モデル、すなわち10-fold CVを2 seedで繰り返したensembleです。CatBoostとTabMはearly stoppingが重要で全データ単体fitが難しく、fold ensembleで全データを活用しました。CVは通常3 seedでしたが、推論Notebookの30 GB RAMのうちtree searchが18 GB、modelが10 GBを使うため2 seedだけ採用。seedは恣意的選択で、調整していません。

「CVを信じる」対「LBを信じる」

最終週は2戦略を並行しました。

1. **Trust LB:** extra data v4（最終の約半量）で偶然LBが良かったOptuna候補を選び、ensemble重みもLBへ調整。feature selection・augmentationは不完全。
2. **Trust CV:** Optuna候補を10-fold・複数seedで再検証し、extra data v6でaugmentation、feature selection、追加データweightをCVへ調整。終盤12時間または終了後までLB未検証。

予想外にもTrust CVがPublic・Private双方で勝ちました。Public LBへ調整したensembleがCV重みensembleより強く見え、追加データ倍増も初期にはLBを悪化させ、CV-Public相関も弱かったため驚きました。Trust CVは大shake-up回避用で、Public調整戦略を上回る想定ではありませんでした。

Strategy	CV	Public	Private
Trust LB	0.3554	0.4142	0.4190
Trust CV	0.3442	0.4137	0.4178

あまり機能しなかったこと

未分割agent文字列のカテゴリ利用、TF-IDF/LSA、MLP、XGBoost、text embedding、ruleから均衡度を予測するLM、OpenFE、公開Notebookの追加特徴量、OOF stacking、importance・forward/backward feature selection、MCTS config部品ごとの平均Elo推定、極端なwin/loss/draw game classifier、random noise augmentation、CatBoost monotonic constraint、CatBoost finetuning、複数MCTS algorithmによる初期局面評価。追加努力で機能するものもあるでしょうが、初期結果が悪い案からは早めに移りました。

見落とした改善

ほぼ全上位チームが使ったflip/inversion augmentationを完全に見落としました。これが他チームがtree searchなしでも迫れた主因でしょう。7位解法の数値が正しければhybridはPrivate約0.407、私の解法より0.01良くなり得ます。

Links

- [Inference notebook](#)
- [Extra training data](#)
- [Starting-position analysis](#) • [training](#) • [test](#) • [generation code](#)

補足Q&A

Robert Hatch: LightGBMのDart固有hyperparameterはdefault以外を使いましたか？何度か試しましたが収束せず、方向性も分からず諦めています。

James Day: はい、複数のDart固有parameterを調整しました。ただしDart固有parameterを調整する前でも良い結果で、通常parameterだけ20 trialほど再調整したconfigでもdefault boostingを上回りました。

Geremie Yeo: OOF予測を特徴量とするisotonic regression後処理が興味深いです。私は正負方向それぞれのquantile loss modelを使い、元予測と混ぜましたが単純scaleと同程度でした。同様の案を試し、isotonicより良かったですか？

James Day: 試していません。scale、任意clip、通常／centered isotonicだけです。あなたの方法はscale程度、私ではscaleがscale+clipより悪く、LB調整scale+clipと無調整isotonicが同程度だったため、quantile法はisotonicより悪い可能性があります。ただしraw予測分布の違いもあり得ます。後処理の目的は絶対値を増やすだけではありません。最適clipは約-0.96～0.96で、isotonicはsigmoid/tanhに似ており、極端なraw予測は外側でなく内側へscaleする必要がありました。

Ruby: test上でLudiiを使うsimulation-based featureは、主催者がsimulationを禁止した投稿に反しませんか？

James Day: 心配しましたが、(1) Ludiiをsimulation実行には使っていない、(2) Ludii自体を改変せずdependencyとしてlinkし、CC BY-NC-ND 4.0 に反せず配布できる、(3) rules tabの正式ruleに反しない、ため問題ないと判断しました。質問すると手法が露見することを恐れました。依存部分をC++へ書き直すことも考えましたが時間がありませんでした。

Mart Preusse: tree searchでtrue advantageをどう推定したのか理解できませんでした。

James Day: inference Notebookの隠れたdependencyです。AnalyzeGameStartingPosition_v3.jar と Ludii-1.3.13.jar を展開すれば読めますがcompileで崩れています。読みやすいcodeを公開予定です。LudiiのMCTSはmove選択時に誰が勝っているかを示すutilityを暗黙に計算します。そこを取り出して均衡度としました。開始前から大差でwin/lossと判断するなら、そのゲームは不均衡でしょう。

Vasilis: n_estimators が非常に大きい理由は？通常300~500に固定していました。

gezi: TabMではtree leaf特徴量またはOOF予測を使いましたか？tree特徴量なしで高いCV/LBになりますか？また centered isotonicは $w \cdot x + b$ より約0.0005良くなりました。

James Day: TabMはleaf特徴量もOOFも使わず、そのCV/LBを達成しました。後処理を除けば純DLです。CIRの強さが私の予測固有でない確認にもなりました。

James Day: LightGBMはDartとearly stoppingを併用できず、最終LightGBMではearly stoppingなしです。n_estimators を調整すれば精度面で問題ないと考えました。Optunaの最良値が探索端ならrangeを広げ、empirical rangeを反復修正しました。300~500はこのdatasetには少なすぎ、良かったLGBMは5,586~26,412、探索上限は35,000でした。

Dennis Soemers (ホスト) : この使い方は問題ありません。禁止の趣旨はtest labelの近似を生成するsimulationでした。test label自体がsimulation結果なので、hidden test・batch API・短い時間制限で正確なlabel生成を防ぎました。この処理はlabel近似でなく追加特徴量の生成で、15秒という短いsearchです。未知gameへ最適algorithmを素早く決める研究目的にも合い、数時間の全algorithm simulationとは異なります。